

AI Audit Report — Phụ lục bắt buộc

HW06 — API Testing · Sinh viên: 23127195 · Ngày: 2026-09-01

1. Khai báo

"I use AI tools for the following tasks."

Tôi có sử dụng công cụ AI trong bài tập này. Toàn bộ quá trình sử dụng được ghi lại đầy đủ trong tài liệu này và trong nhật ký chi tiết

[interactions/SESSION-01_2026-09-01.md](#).

2. Công cụ đã dùng

Công cụ	Phiên bản / Model ID	Dùng cho việc gì
Claude Opus 5	claude-opus-5, qua Claude Code CLI / tiện ích VS Code	Phân tích đặc tả, sinh test case, viết mã hạ tầng kiểm thử, soạn tài liệu
Postman	12.26.1 (desktop, Windows)	Thiết kế và chạy collection, Console lấy bằng chứng header
Newman	6.2.2 + newman-reporter-htmlextra 1.23.1	Thi hành tự động, xuất báo cáo HTML/JSON/JUnit
Node.js	v22.20.0	Chạy SUT và Newman
Python	3.12 (+ openpyxl)	Bộ biên dịch IR → Postman, xuất Excel
curl	(MinGW)	Tái hiện lỗi độc lập với Postman
GitHub Actions	ubuntu-latest	CI/CD

Chế độ làm việc: AI chạy ở chế độ *agentic* — có quyền đọc/ghi file trong repo, chạy lệnh shell, và gọi HTTP tới SUT chạy trên `localhost:3000`. Điều này có nghĩa AI không chỉ *đề xuất* mà còn *thực thi*; do đó phần rà soát của con người tập trung vào việc **đối chiếu kết quả AI báo cáo với bằng chứng thật** (log Newman, output `curl`, nội dung file).

3. Bảng tổng hợp các lượt tương tác

Nhật ký đầy đủ (prompt nguyên văn, suy luận của AI, output thật, phán quyết review) nằm ở

[interactions/SESSION-01_2026-09-01.md](#).

Thư viện prompt theo từng bước nằm ở [prompts/PROMPT_LIBRARY.md](#).

#	Thời điểm	Nhiệm vụ	Đầu ra của AI	Phán quyết của SV
INT-01	18:35	Bóc tách đề bài từ PDF	Tóm tắt 17 mục + xác định các ràng buộc cứng	✅ Chấp nhận
INT-02	18:44	Chọn 3 API không trùng nhóm	Đề xuất FR-03 / FR-09 / FR-15	⚠️ Phải chọn lại (xem INT-06)
INT-03	18:46	Dựng môi trường SUT + Newman	SUT chạy localhost:3000, DB reset mỗi lần khởi động	✅ Chấp nhận
INT-04	18:49	Probe curl xác minh lỗi	10 probe, phát hiện 8 lỗi ứng viên	✅ Chấp nhận, 1 sửa (hạ mức user enumeration)
INT-05	18:54	Thiết kế hạ tầng IR + builder	Kiến trúc 2 tầng IR → Postman	✅ Chấp nhận, yêu cầu bổ sung trường audit
INT-06	19:20	Chọn lại API sau khi biết nhóm lấy thêm FR-03/08/15	FR-04 / FR-09 / FR-16	✅ Chấp nhận
INT-07	19:22	Probe FR-04 và FR-16	Xác nhận 8 lỗi mới, gồm 2 Critical	✅ Chấp nhận
INT-08	19:30	Sinh + audit + mở rộng test case (3 API)	144 test case (110 AI + 34 người)	✅ Chấp nhận sau khi rà từng nhãn audit
INT-09	20:05	Thi hành Newman lần 1	144 case, phát hiện 2 khiếm khuyết của chính bộ test	⚠️ Bắt buộc sửa — xem §4
INT-10	20:07	Sửa harness và chạy lại	52 FAIL, tất cả đều là lỗi thật	✅ Chấp nhận
INT-11	20:11	Báo cáo 24 lỗi + script tái hiện	BUG_REPORTS.md, reproduce_bugs.sh	✅ Chấp nhận sau khi tự chạy lại script
INT-12	20:30	CI/CD 2 tầng + Agent Skill	Workflow, generator.py, DESIGN.md	✅ Chấp nhận; diagram tự vẽ (§11)

4. Những chỗ AI sai và đã được sửa

Đây là phần quan trọng nhất của báo cáo audit. Bốn nhóm sai sót được phát hiện:

4.1 — Assertion bất đồng bộ nuốt mất lỗi (ngghiêm trọng nhất)

AI sinh assertion bất đồng bộ theo mẫu:

```
pm.test('...', function (done) {
  pm.sendRequest(opts, function (err, res) {
    pm.expect(res.json().role).to.eql('user');    // ném lỗi -> done() không chạy
    done();
  });
});
```

Khi assertion **đạt**, `done()` chạy → Newman ghi PASS. Khi assertion **hỏng**, ngoại lệ được ném trước `done()` → Newman **âm thầm bỏ qua** test đó: không PASS, không FAIL, biến mất khỏi báo cáo.

Hậu quả thật: TC-A1-037 — test cho BUG-A1-01 (leo quyền lên admin, lỗi nghiêm trọng nhất của cả bài) — ban đầu hiện ra như "đã pass".

Cách phát hiện: đối chiếu số liệu báo cáo Newman với kết quả `curl` thủ công. `curl` cho thấy `role = admin` trong khi báo cáo Newman không có dòng FAIL nào tương ứng. Sự vắng mặt của một assertion — chứ không phải một assertion sai — mới là dấu hiệu.

Đã sửa: bọc `try { ... done(); } catch (e) { done(e); }` cho **32 assertion**. Vì mọi assertion đều sinh từ một khuôn mẫu dùng chung nên chỉ cần sửa một chỗ. Bộ kiểm tra IR nay **chặn** mọi assertion bất đồng bộ thiếu try/catch.

4.2 — Dữ liệu chuẩn bị đặt trong pre-request script

AI dùng `pm.sendRequest` trong pre-request script để chụp mốc số lượng bản ghi (`countBefore`). Cách này không đảm bảo hoàn tất trước khi request chính được gửi.

Hậu quả: TC-A3-001, TC-A3-007, TC-A3-035 FAIL với thông báo `expected 6 to deeply equal NaN` — **ba kết quả FAIL giả**, và TC-A3-032 so sánh với mốc cũ (`10` thay vì `129`).

Đã sửa: thay bằng **26 request [SETUP] tường minh** — tuần tự, quan sát được trong báo cáo, và tự nó cũng có assertion.

4.3 — Biến môi trường rỗng đề lên biến collection

AI khai báo `adminToken`, `userToken` với giá trị rỗng trong file environment, trong khi test script ghi token vào `collection scope`. Trong Postman, **Environment có độ ưu tiên cao hơn Collection**, nên giá trị rỗng luôn thắng.

Hậu quả: toàn bộ nhóm test sau bước đăng nhập trả `401` ở lần chạy đầu tiên.

Đã sửa: bỏ hẳn các biến runtime khỏi file environment, kèm ghi chú giải thích trong file.

4.4 — Kỳ vọng quá chặt so với điều đặc tả thực sự nói (9 test case)

AI có xu hướng ràng buộc `400` cho mọi đầu vào "trông có vẻ sai", kể cả khi tài liệu không quy định. Ví dụ: độ dài tối đa của họ tên (SRS không nêu), số thực cho `total_amount` (không bị cấm), khoảng trắng bao quanh mã giảm giá (không quy định trim).

Đã sửa: 9 case được gắn nhãn `INCOMPLETE` và nói kỳ vọng thành `statusIn [...]` hoặc "không được 5xx", kèm ghi chú nêu rõ *tài liệu im lặng ở điểm này*.

Ngoài ra, TC-A1-002 (user enumeration ở phương án API cũ) bị AI xếp là "vi phạm đặc tả"; sinh viên hạ xuống mức *sai lệch so với thông lệ bảo mật (OWASP ASVS 2.5)* vì SRS **không** phát biểu tường minh yêu cầu này.

5. Số liệu kiểm toán

Chỉ số	Giá trị
Tổng test case	144
Do AI sinh	110 (76,4 %)
Do sinh viên tự thêm	34 (23,6 %)
Test case AI được gán <code>VALID</code>	101 / 110 (91,8 %)
Test case AI được gán <code>INCOMPLETE</code> và đã hiệu chỉnh	9 / 110 (8,2 %)
Test case AI bị gán <code>INVALID</code> và loại bỏ	0
Khiếm khuyết của chính bộ test do AI tạo ra	3 (\$4.1, \$4.2, \$4.3)
Lỗi thật của SUT tìm được	24
Lỗi chỉ tìm được nhờ test case do người viết	6 / 24 (25 %)

Con số đáng chú ý nhất không phải 91,8 % nhãn `VALID` , mà là **3 khiếm khuyết ở tầng hạ tầng**. Ở mức từng test case, AI làm tốt. Ở mức *khuôn mã dùng chung* — nơi một lỗi nhân lên 32 lần và lại **giấu** kết quả thay vì báo sai — AI mắc lỗi có hệ thống. Xem `AI_CRITIQUE.md` .

6. Những phần KHÔNG dùng AI

Theo §11 (Anti-AI-Cheat) của đề bài, các hạng mục sau bắt buộc do người thật thực hiện:

§11 liệt kê **đích danh ba thứ** không được để AI sinh: ảnh chụp Postman Console chứng minh header `X-Student-Id` , output Newman, và sơ đồ bộ sinh test case.

Hạng mục §11	Trạng thái
Sơ đồ bộ sinh test case	✅ Sinh viên tự vẽ bằng draw.io. Repo kèm file nguồn <code>.drawio</code> mở lại được để kiểm chứng, không chứa bản vẽ nào do AI sinh
Ảnh chụp Postman Console	✅ Sinh viên tự chụp , 3 ảnh trong <code>evidence/</code>
Newman run output	✅ Chạy thật trên <code>localhost:3000</code> — báo cáo trong <code>newman/</code>
Video demo YouTube	⚠️ Sinh viên phải tự quay

6.1 — Ảnh chụp màn hình cho 24 GitHub Issue (có dùng AI, khai báo theo AI Policy)

Hạng mục này **không** nằm trong danh sách cấm của §11, và AI Policy của bài là *Open* kèm điều kiện khai báo. Khai báo cụ thể:

Thành phần	Ai làm	Ghi chú
Dữ liệu trong ảnh	Sinh viên	Output của lần chạy thật <code>bash bugs/reproduce_bugs.sh</code> trên SUT <code>localhost:3000</code> . Lưu tại <code>bugs/evidence/reproduce_output.txt</code>
Cắt theo từng mã lỗi	AI	<code>bugs/split_evidence.py</code> + <code>bugs/chunk_evidence.py</code> — chỉ cắt, không sửa nội dung
Thao tác chụp	AI	<code>bugs/capture_screenshots.ps1</code> mở cửa sổ Windows Terminal hiển thị bằng chứng rồi chụp toàn màn hình bằng <code>Graphics.CopyFromScreen</code>
Máy thực thi	Sinh viên	Chụp trên chính máy làm bài; ảnh BUG-A3-09 hiện đường dẫn thật <code>D:\Kiem_thu\HW6\.sut\...\server.js:214:14</code>

Ảnh là ảnh chụp màn hình thật, không phải ảnh do AI vẽ lại: script đọc pixel trực tiếp từ màn hình, đúng cơ chế mà Snipping Tool dùng. Từng ký tự trong ảnh đối chiếu được với file văn bản tương ứng trong `bugs/evidence/per_bug/`.

Mỗi ảnh cho thấy lệnh `curl` đầy đủ rồi mới đến response. Đây là kết quả của một vòng review: bản đầu tiên của `reproduce_bugs.sh` in ra dòng mô tả lệnh viết tay bằng `echo` (ví dụ `echo '$ curl -X PUT /api/users/me -d {...}'`) trong khi lệnh thật chạy ở dòng dưới với biến `"${H[@]}"` `"${AU[@]}"` mà người đọc không nhìn thấy. Dòng hiển thị và lệnh thật là hai thứ khác nhau — một dạng bằng chứng không kiểm chứng được. Script đã được viết lại quanh hàm `run()`: nó nhận lệnh dưới dạng mảng đối số, dựng dòng hiển thị **từ chính mảng đó**, rồi chạy bằng `"$@"`. Nhờ vậy thứ in ra luôn đúng bằng thứ được chạy, và người chấm copy lại chạy được ngay.

Một phương án đã bị loại bỏ. Ban đầu có thử dựng ảnh bằng thư viện đồ hoạ (Pillow) từ chính văn bản đó. Dữ liệu vẫn thật, nhưng ảnh được vẽ thêm thanh tiêu đề và ba chấm tròn khiến nó *trông như* ảnh chụp cửa sổ trong khi không phải. Phương án này bị bỏ vì hình thức ngụ ý sai về nguồn gốc, dù nội dung không sai. Script đó đã xoá khỏi repo.

Bản đầy đủ của nhật ký tương tác: [interactions/SESSION-01_2026-09-01.md](#)

Phần phê bình AI (bắt buộc, 200–300 từ): [AI_CRITIQUE.md](#)